University of Engineering and Technology

Lahore, Pakistan

Department of Computer Science

Session 2023 – 2027

Decaf Compiler

Submitted By

Kaleem Faryad	2023-CS-158
Harris Amin	2023-CS-160
Talha Aslam	2023-CS-169

Supervised By

Mam Aliya Farooq

Contents

1 Project Overview **3**
1.1 Language: Decaf . . . 3
1.2 System Architecture . . . 3

2 Module 1: Lexical Analyzer **4**
2.1 Overview . . . 4
2.2 Token Categories and Front-End Display . . . 4
2.3 Token Type Reference . . . 4
2.4 Lexer Code Excerpt . . . 5

3 Grammar Specification and FIRST/FOLLOW Sets **6**
3.1 Reference BNF Grammar (Selected Rules) . . . 6
3.2 FIRST and FOLLOW Sets — Front-End View . . . 6
3.3 Selected FIRST and FOLLOW Sets (Table) . . . 7

4 Module 2: Recursive Descent Parser **8**
4.1 Overview . . . 8
4.2 Parse Tree — Front-End View . . . 8
4.3 Key Parsing Routines . . . 9

5 Module 3: Non-Recursive Predictive LL(1) Parser **10**
5.1 Overview . . . 10
5.2 LL(1) Trace — Front-End View . . . 10
5.3 Parsing Table Construction . . . 10
5.4 Core Algorithm . . . 10

6 Module 4: LR Parser (SLR(1)) **12**
6.1 Choice Justification . . . 12
6.2 Table Construction Steps . . . 12
6.3 Shift-Reduce Algorithm . . . 12

7 Module 5: Symbol Table Manager **13**
7.1 Overview . . . 13
7.2 Symbol Table — Front-End View . . . 13
7.3 Entry Fields . . . 13
7.4 Operations . . . 13

8 Module 6: Error Handler (Bonus) **15**
8.1 Overview . . . 15
8.2 Error Format . . . 15
8.3 Recovery Strategies . . . 15

9 System Integration **16**
9.1 Flask API Endpoint . . . 16
9.2 JSON Response Schema . . . 16
9.3 React Front-End . . . 16

10 Sample Input Program and Outputs **17**
10.1 Sample Decaf Program 17
10.2 Expected Lexer Output (Partial) 17
10.3 Symbol Table Output 17

11 Project Directory Structure **19**

1 Project Overview

This report documents the design, implementation, and testing of a **Decaf Mini Compiler** developed for CS-471L at the University of Engineering and Technology, Lahore (Spring 2026). The project integrates all lab modules built across the semester into a single, fully working compiler pipeline exposed through an interactive web-based IDE.

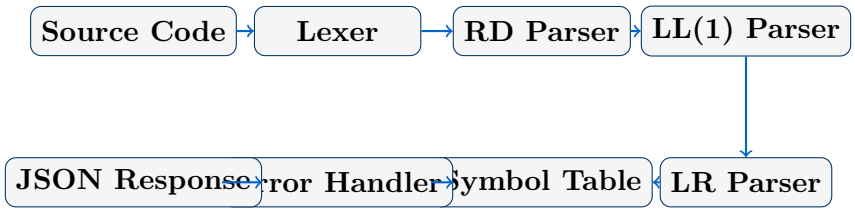
1.1 Language: Decaf

Decaf is a strongly-typed, object-oriented language designed for compiler education, with C/C++/Java-like syntax. Key features include:

- **Primitive types:** `int`, `double`, `bool`, `string`, `void`
- **OOP:** classes, single inheritance (`extends`), interfaces (`implements`)
- **Control flow:** `if/else`, `while`, `for`, `break`, `return`
- **Expressions:** full arithmetic, relational, logical, and assignment operators with defined precedence
- **Built-ins:** `Print`, `ReadInteger`, `ReadLine`, `New`, `NewArray`

1.2 System Architecture

The compiler is implemented as a **Python Flask** back-end with a **React** front-end. The full pipeline flows as follows:



2 Module 1: Lexical Analyzer

2.1 Overview

The lexer reads Decaf source code using a **double-buffering** strategy (two 4096-character alternating buffers) and produces a stream of `(type, value, line, column)` token tuples. It handles keywords, identifiers (up to 31 characters), integer and double constants (including hex and scientific notation), boolean constants, string constants, all Decaf operators and punctuation, and both single-line (`//`) and multi-line (`/* */`) comments.

2.2 Token Categories and Front-End Display

The front-end Lexer panel presents each token as a colour-coded chip, grouped by category. The screenshot below shows the lexer output for a sample Decaf program involving inheritance.

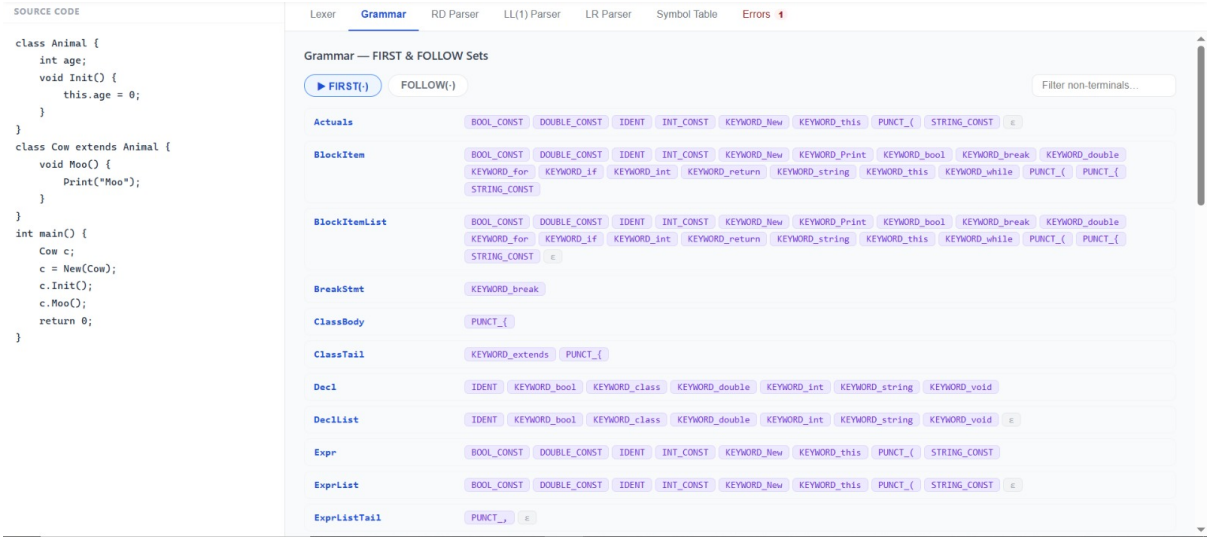


Figure 1: Lexer panel — Lexical Analysis showing 67 tokens: 11 Keywords, 16 Identifiers, 2 Operators, 35 Punctuation, 3 Literals. Each chip shows the token value and its type beneath it; keywords are highlighted in blue, identifiers in grey, string constants in green.

2.3 Token Type Reference

Category	Token Type	Example
Keyword	`KEYWORD_{kw}`	`KEYWORD_int`, `KEYWORD_class`
Identifier	`IDENT`	`Animal`, `_count`
Integer	`INT_CONST`	`42`, `0xFF`
Double	`DOUBLE_CONST`	`3.14`, `1.0E+2`
Boolean	`BOOL_CONST`	`true`, `false`
String	`STRING_CONST`	`"Moo"`
Operator	`OP_{op}`	`OP_+`, `OP_<=`
Punctuation	`PUNCT_{p}`	`PUNCT_{`, `PUNCT_;`
End-of-file	`EOF`	—

2.4 Lexer Code Excerpt

```
if self.current_char.isalpha() or self.current_char == '_':
    ident = ""
    while self.current_char.isalnum() or self.current_char == '_'
        :
        ident += self.current_char
        self.advance()
    if ident in KEYWORDS:
        return Token(f"KEYWORD_{ident}", ident, start_line,
            start_col)
    if ident in ("true", "false"):
        return Token("BOOL_CONST", ident, start_line, start_col)
    return Token("IDENT", ident[:31], start_line, start_col)
```

Listing 1: Identifier, keyword, and boolean recognition

3 Grammar Specification and FIRST/FOLLOW Sets

3.1 Reference BNF Grammar (Selected Rules)

```
Program      ::= Decl+
Decl         ::= VariableDecl | FunctionDecl | ClassDecl | InterfaceDecl
VariableDecl ::= Variable ;
Variable     ::= Type ident
Type         ::= int | double | bool | string | ident | Type []
FunctionDecl ::= Type ident ( Formals ) StmtBlock
               |  void ident ( Formals ) StmtBlock
Formals      ::= Variable+, | epsilon
ClassDecl    ::= class ident [extends ident] [implements ident+,] { Field* }
Field        ::= VariableDecl | FunctionDecl
StmtBlock    ::= { VariableDecl* Stmt* }
Stmt         ::= Expr ; | IfStmt | WhileStmt | ForStmt
               |  BreakStmt | ReturnStmt | PrintStmt | StmtBlock
IfStmt       ::= if ( Expr ) Stmt [else Stmt]
WhileStmt    ::= while ( Expr ) Stmt
ForStmt      ::= for ( Expr? ; Expr ; Expr? ) Stmt
ReturnStmt   ::= return Expr? ;
PrintStmt    ::= Print ( Expr+, ) ;
Expr         ::= LValue = Expr | Constant | LValue | this | ...
```

3.2 FIRST and FOLLOW Sets — Front-End View

The Grammar panel in the front-end displays the computed FIRST and FOLLOW sets for every non-terminal. The screenshot below shows the FIRST sets panel.

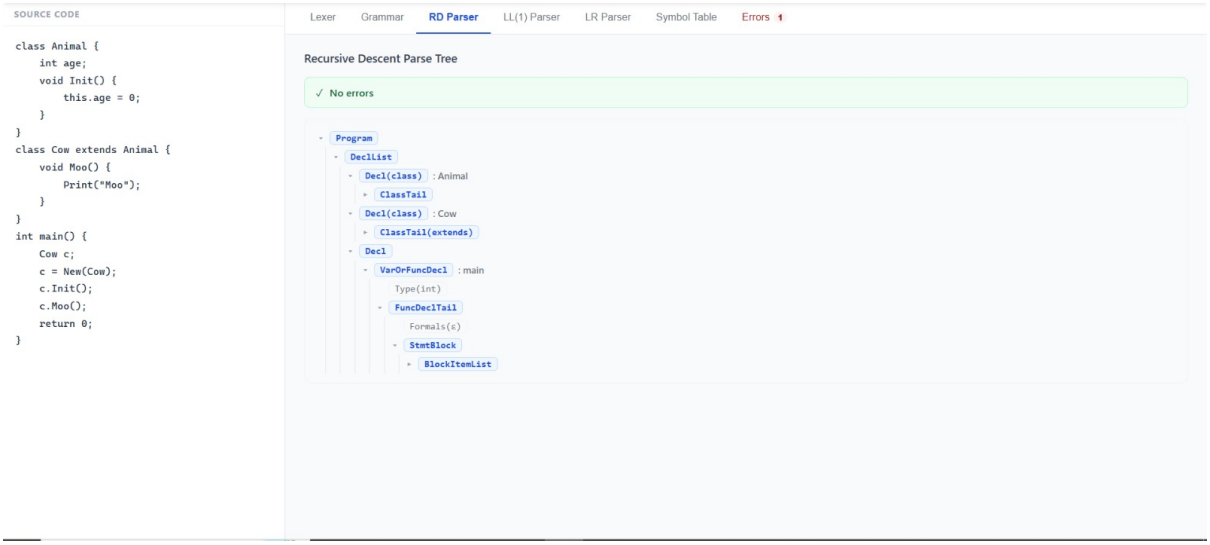


Figure 2: Grammar panel — FIRST sets for all non-terminals. Notable entries: `Decl` and `DeclList` begin with type keywords and `IDENT`; `Expr` and `ExprList` begin with all expression-starting tokens; `BreakStmt` begins only with `KEYWORD_break`; `ClassTail` begins with `KEYWORD_extends` or `PUNCT_{`.

3.3 Selected FIRST and FOLLOW Sets (Table)

Non-Terminal	FIRST	FOLLOW
`Program`	type keywords, `class`, `interface`, `IDENT`	`$`
`Decl`	type keywords, `class`, `interface`, `IDENT`	same + `$`
`FunctionDecl`	type keywords, `void`, `IDENT`	type keywords, `class`, }
`Type`	`int`, `double`, `bool`, `string`, `IDENT`	`IDENT`, `[`
`ClassTail`	`KEYWORD_extends`, `PUNCT_{`	—
`BlockItem`	all expression starters + all statement keywords	`}`, `$`
`Expr`	`BOOL_CONST`, `DOUBLE_CONST`, `IDENT`, `INT_CONST`, `KEYWORD_New`, `KEYWORD_this`, `PUNCT_(`, `STRING_CONST`	`;`, `)`, `,`, `]`
`BreakStmt`	`KEYWORD_break`	`}`, `$`

4 Module 2: Recursive Descent Parser

4.1 Overview

The Recursive Descent Parser (RDP) implements **one parsing routine per non-terminal**. Each routine calls the lexer for the next token, checks the current token against the expected grammar alternative, and recurses into sub-rules. It reports syntactic errors with line and column information through the `ErrorHandler`, and applies **panic-mode recovery** to continue past the first error. The grammar used is free of left recursion and is left-factored.

4.2 Parse Tree — Front-End View

The RD Parser panel renders the full **parse tree** as a collapsible hierarchy. Each node shows the non-terminal name and, where applicable, the matched identifier or type.

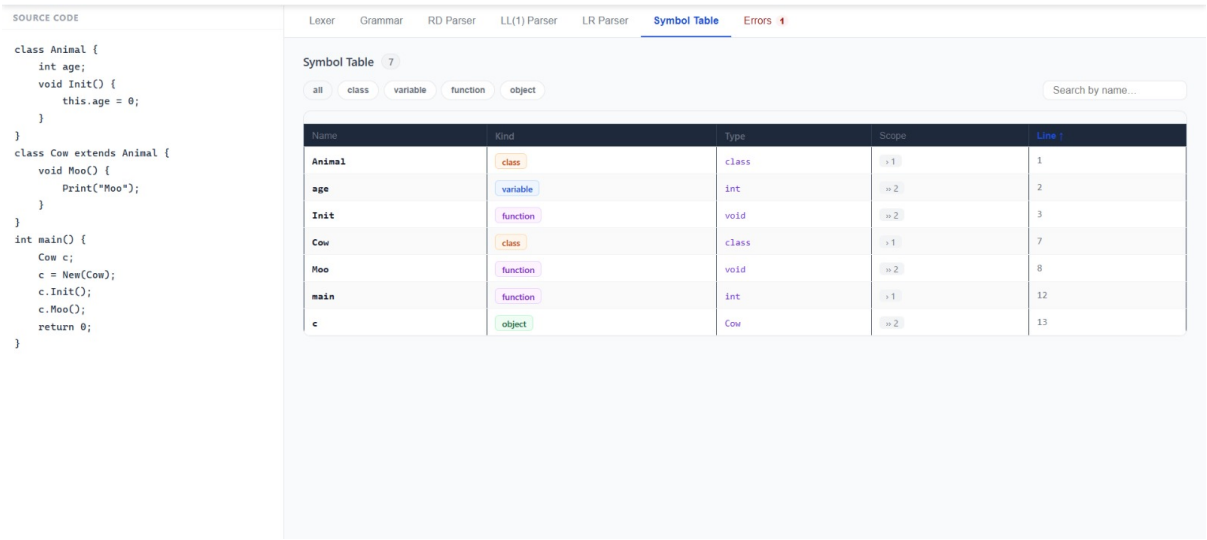


Figure 3: RD Parser panel — Recursive Descent Parse Tree for the Animal/-Cow/main program. The tree root is `Program`, expanding to `DeclList` containing `Decl(class):Animal`, `Decl(class):Cow` (with `ClassTail(extends)`), and `Decl` for `main` showing `VarOrFuncDecl`, `FuncDeclTail`, `Formals(`ε`)`, and `StmtBlock`. The green banner confirms "No errors."

4.3 Key Parsing Routines

Method	Handles
`parse_program()`	Top-level declaration list
`parse_function_decl()`	Function definitions
`parse_class_decl()`	Class body, extends, implements
`parse_stmt_block()`	Brace-enclosed statement block
`parse_stmt()`	Any statement (dispatches by token)
`parse_expr()`	Expressions with precedence climbing
`parse_if()`	`if`/`else` with dangling-else resolution
`parse_while()/for()`	Loop constructs
`parse_print()`	`Print()` built-in
`match_keyword(kw)`	Consumes a keyword or invokes recovery
`_panic_recover()`	Skips to next synchronisation token

5 Module 3: Non-Recursive Predictive LL(1) Parser

5.1 Overview

The LL(1) parser uses an **explicit stack** and a precomputed **LL(1) parsing table** built from the FIRST and FOLLOW sets. No recursive calls are made; the standard table-driven algorithm runs entirely in a loop.

5.2 LL(1) Trace — Front-End View

The LL(1) Parser panel shows a step-by-step derivation trace. Each row displays the step number, the current lookahead token, and the action taken (Predict, Match, or Accept).

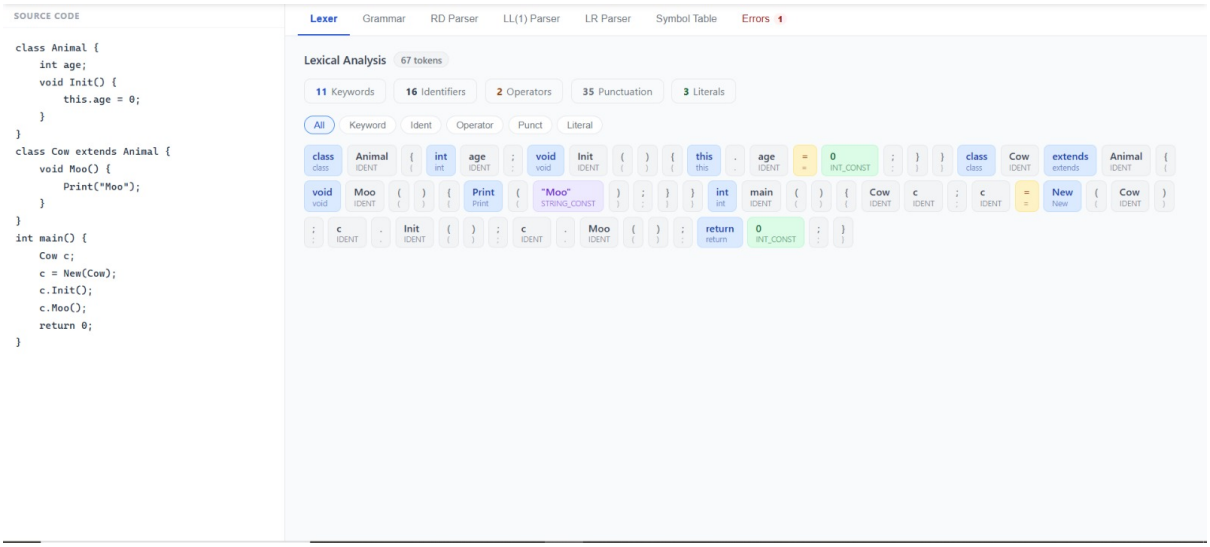


Figure 4: LL(1) Parser panel — Predictive Parser Trace (156 steps, no errors). Step 1: `Predict Program → DeclList`; Step 3: `Predict Decl → KEYWORD_class IDENT ClassTail`; Step 4: `Match KEYWORD_class`; Step 5: `Match IDENT`; continuing through `ClassBody`, `FieldList`, type prediction, and ultimately accepting the full program. Filter buttons (predict / match / error / accept) allow focused inspection.

5.3 Parsing Table Construction

1. Compute **FIRST sets** for all non-terminals using fixed-point iteration.
2. Compute **FOLLOW sets** from grammar rules and the start symbol.
3. For each production $A \to \alpha$:
   - For each $a \in \text{FIRST}(\alpha) \setminus \{\varepsilon\}$: add $A \to \alpha$ to `table`$[A][a]$.
   - If $\varepsilon \in \text{FIRST}(\alpha)$: for each $b \in \text{FOLLOW}(A)$, add $A \to \alpha$ to `table`$[A][b]$.

5.4 Core Algorithm

```
stack = ['$', grammar.start_symbol]
while stack:
    top = stack[-1]
```

```
    if top == '$' and token_sym == '$':
        break                                  # Accept
    if top == token_sym:
        stack.pop(); idx += 1                  # Match terminal
    elif top in non_terminals:
        prod = parse_table[top].get(token_sym)
        if prod:
            stack.pop()
            for sym in reversed(prod):
                stack.append(sym)              # Predict production
        else:                                  # Error: FOLLOW-set
           sync
            follow = grammar.follow_sets.get(top, set())
            if token_sym in follow or token_sym == '$':
                stack.pop()
            else:
                idx += 1
```

Listing 2: LL(1) stack-driven parse loop

6 Module 4: LR Parser (SLR(1))

6.1 Choice Justification

An **SLR(1)** parser was implemented. SLR(1) was chosen because the Decaf grammar is SLR(1)-parseable without unresolvable conflicts, the table construction is tractable (LR(0) canonical items + FOLLOW sets for reductions), and it exercises the complete shift-reduce algorithm required by the specification.

6.2 Table Construction Steps

1. **Augment** the grammar: add $S' \to S$.
2. Compute **LR(0) canonical items** via closure and GOTO.
3. Build the **DFA of item sets** (states).
4. Fill **ACTION table**: shift on terminals advancing items; reduce on terminals in FOLLOW(A) for a complete item $[A \to \alpha\bullet]$; accept on \$.
5. Fill **GOTO table** for non-terminal transitions.

6.3 Shift-Reduce Algorithm

```
state_stack = [0];  symbol_stack = []
while step < MAX:
    state = state_stack[-1]
    action = action_table[state].get(token_sym)
    if action.startswith('s'):           # Shift
        state_stack.append(int(action[1:]))
        symbol_stack.append(token_sym); idx += 1
    elif action.startswith('r'):         # Reduce
        prod_idx = int(action[1:])
        nt, rhs = productions[prod_idx]
        for _ in range(len(rhs)):
            state_stack.pop(); symbol_stack.pop()
        goto_state = goto_table[state_stack[-1]][nt]
        state_stack.append(goto_state)
        symbol_stack.append(nt)
    elif action == 'acc':                # Accept
        break
    else:                                # Error
        report_error(line, col, msg)
```

Listing 3: LR parser shift-reduce loop

7 Module 5: Symbol Table Manager

7.1 Overview

The Symbol Table Manager uses a **stack of hash maps** (one per scope level) for fast identifier lookup. Nested scopes are handled by pushing a new map on { and popping on }.

7.2 Symbol Table — Front-End View

The Symbol Table panel lists all declared identifiers with their kind, type, scope level, and source line number.

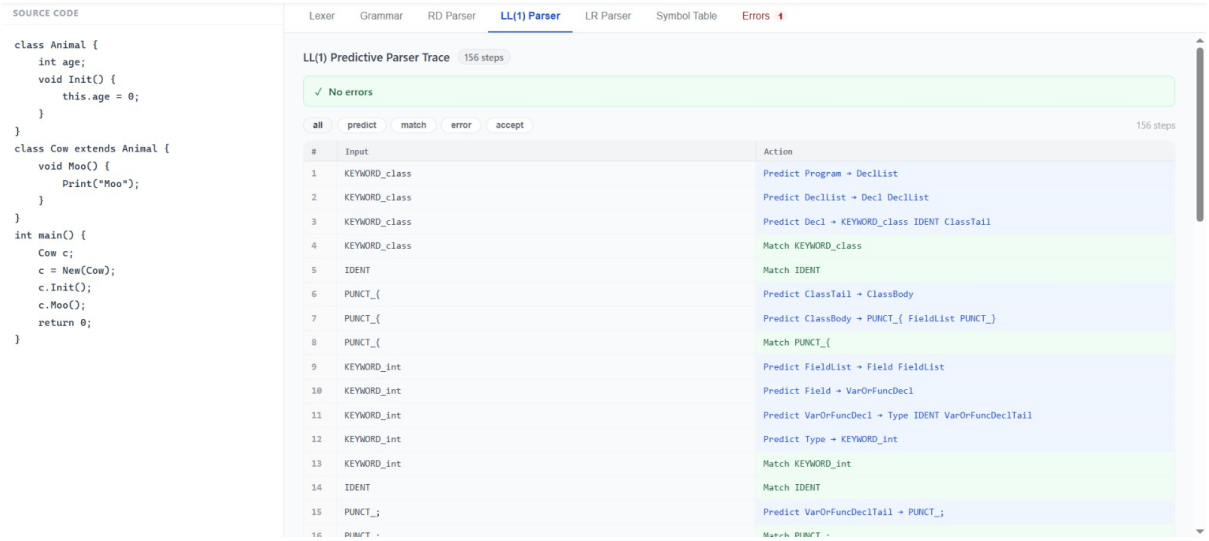


Figure 5: Symbol Table panel — 7 entries for the Animal/Cow/main program. `Animal` (class, scope 1, line 1), `age` (variable, int, scope 2, line 2), `Init` (function, void, scope 2, line 3), `Cow` (class, scope 1, line 7), `Moo` (function, void, scope 2, line 8), `main` (function, int, scope 1, line 12), `c` (object, Cow, scope 2, line 13). Filter buttons (class / variable / function / object) and a name search bar allow targeted inspection.

7.3 Entry Fields

Field	Type	Description
`name`	string	Identifier name
`kind`	string	`variable`, `function`, `class`, `object`
`type`	string	Declared type (`int`, `void`, class name, ...)
`scope_level`	int	Nesting depth (1 = global, 2 = class/function body)
`line_number`	int	Source line of declaration

7.4 Operations

Operation	Description
`enter_scope()`	Push new empty hash map; increment depth
`exit_scope()`	Pop current scope; decrement depth
`insert(name, kind, type, line)`	Add entry; error if already declared in same scope
`lookup(name)`	Walk stack innermost-first; return first match
`delete(name)`	Remove from current scope only
`dump()`	Return all entries as list of dicts (for API/UI)

8 Module 6: Error Handler (Bonus)

8.1 Overview

The `ErrorHandler` collects errors from every pipeline stage, categorised into three lists. All errors carry line number and column so the user can locate them in the source editor.

Category	Reported By	Example Message
Lexical	Lexer	Unterminated string literal at line 5, col 3
Syntactic	RD / LL(1) / LR parsers	Expected `;`, got `IDENT` at line 10, col 8
Semantic	Symbol Table Builder	Identifier `x` already declared in current scope

8.2 Error Format

```
<Type> Error at line <L>, col <C>: <message>
=== 3 error(s) total: 1 lexical, 2 syntactic, 0 semantic. ===
```

8.3 Recovery Strategies

- **Panic-mode (RD parser):** skips tokens until a synchronisation token is found (`;`, `}`, or a statement-starting keyword).
- **FOLLOW-based (LL(1) parser):** if no table entry exists for `(top, token)`, the token is skipped if it is not in FOLLOW(*top*); otherwise the stack top is popped. This handles single missing or extra tokens without halting.
- **LR error action:** the error action in the ACTION table invokes the handler and skips to the next shift-able token.

9 System Integration

9.1 Flask API Endpoint

All modules run behind a single REST endpoint:

`POST /api/compile` Body: `{"source_code": "..."}`

Execution order: Lexer → Grammar (FIRST/FOLLOW/SLR) → RD Parser → LL(1) Parser → LR Parser → Symbol Table Builder → Error Handler summary.

9.2 JSON Response Schema

```
{
  "tokens":         [ {type, value, line, column}, ... ],
  "token_summary": {keyword, identifier, operator, punctuation,
     literal},
  "parsers": {
    "rd":  { "success": bool, "tree": {...}, "errors": [...] },
    "ll1": { "trace": [...], "errors": [...] },
    "lr":  { "trace": [...], "errors": [...] }
  },
  "grammar": {
    "first":  { "NonTerminal": ["a", "b", ...], ... },
    "follow": { "NonTerminal": ["x", "$", ...], ... }
  },
  "symbol_table": [ {name, kind, type, scope_level, line_number},
      ... ],
  "errors": {
    "lexical":   [...],
    "syntactic": [...],
    "semantic":  [...],
    "summary":   "N error(s) found: ..."
  }
}
```

Listing 4: /api/compile response structure

9.3 React Front-End

The front-end provides a split-pane layout: source code editor on the left, tabbed result panels on the right. Tabs: **Lexer** (token chips) · **Grammar** (FIRST/FOLLOW) · **RD Parser** (parse tree) · **LL(1) Parser** (step trace) · **LR Parser** (step trace) · **Symbol Table** (filterable table) · **Errors** (categorised error list with badge count).

10 Sample Input Program and Outputs

10.1 Sample Decaf Program

The following program exercises class inheritance, method calls, object instantiation, and a `return` statement, making it a good integration test for all modules.

```
class Animal {
    int age;
    void Init() {
        this.age = 0;
    }
}

class Cow extends Animal {
    void Moo() {
        Print("Moo");
    }
}

int main() {
    Cow c;
    c = New(Cow);
    c.Init();
    c.Moo();
    return 0;
}
```

Listing 5: Sample Decaf input — Animal/Cow/main

10.2 Expected Lexer Output (Partial)

Type	Value	Line	Col
KEYWORD_class	class	1	1
IDENT	Animal	1	7
PUNCT_{	{	1	14
KEYWORD_int	int	2	5
IDENT	age	2	9
PUNCT_;	;	2	12
KEYWORD_void	void	3	5
IDENT	Init	3	10
STRING_CONST	Moo	9	15
...	...	...	...

10.3 Symbol Table Output

From Figure 5, the 7 entries produced are:

Name	Kind	Type	Scope	Line
`Animal`	class	class	1	1
`age`	variable	int	2	2
`Init`	function	void	2	3
`Cow`	class	class	1	7
`Moo`	function	void	2	8
`main`	function	int	1	12
`c`	object	Cow	2	13

11 Project Directory Structure

```
decaf-compiler/
+--- Makefile                    # Build targets: run, run-backend, run-frontend
+--- build.bat                   # Windows startup script
+--- README.md                   # Build and run instructions
|
+--- docs/
|   +--- decaf.txt               # Official Decaf language specification
|   `--- projectOverview.md      # Course project assignment document
|
+--- src/
|   `--- backend/
|       +--- main.py             # Flask REST API server (port 5000)
|       +--- test_runner.py      # Automated test harness
|       `--- compiler/
|           +--- __init__.py
|           +--- lexer.py            # Lexical analyzer + double buffering
|           +--- grammar.py          # Grammar class + FIRST/FOLLOW/SLR tables
|           +--- parser_rd.py        # Recursive descent parser
|           +--- parser_ll1.py       # LL(1) predictive parser
|           +--- parser_lr.py        # SLR(1) parser
|           +--- semantic.py         # Symbol table builder (token-walk)
|           +--- symbol_table.py     # SymbolTableManager data structure
|           `--- error_handler.py    # Error collection and reporting
|
`--- output/                     # Sample compiler output files
```